Traffic light recognition

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

1. Learn the actions the car takes after recognizing traffic lights.
2. Learn the newly discovered key source code.

2. Road Sign Detection

Road sign detection source code path:

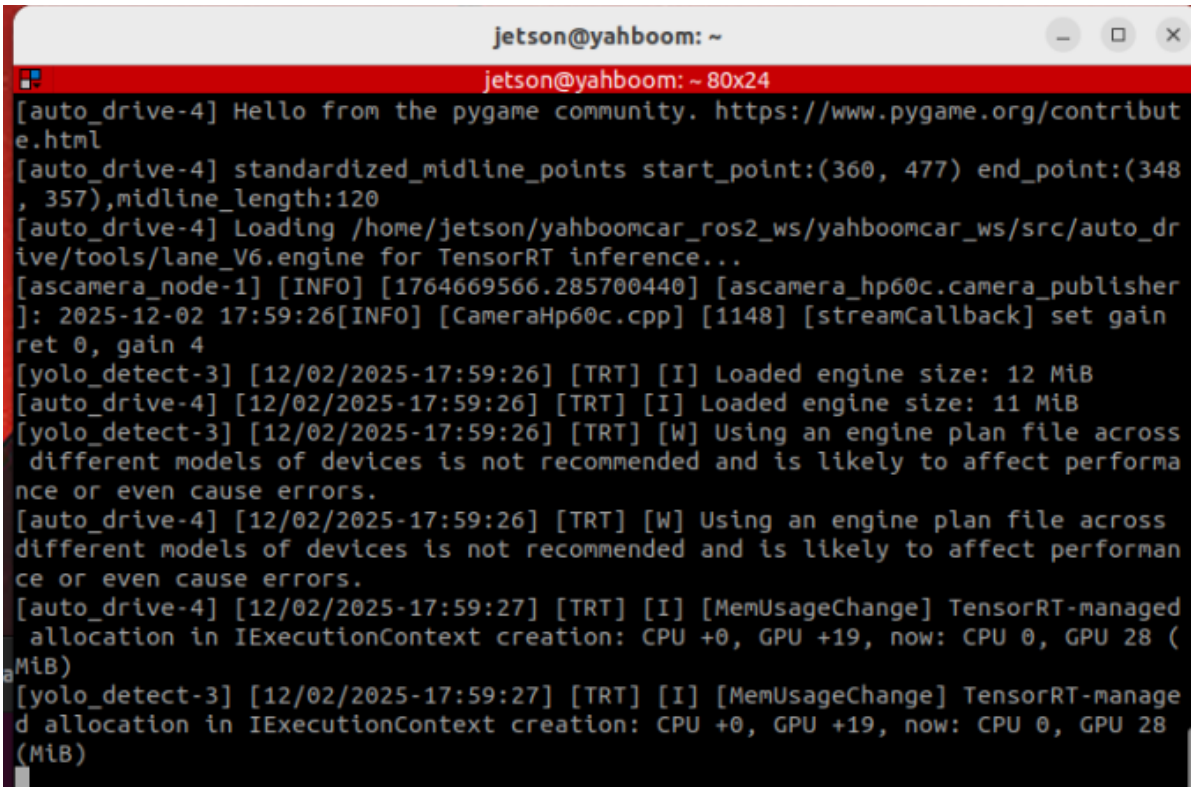
```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/yolo_det
ect.py
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_dri
ve.py
```

3. Program Startup

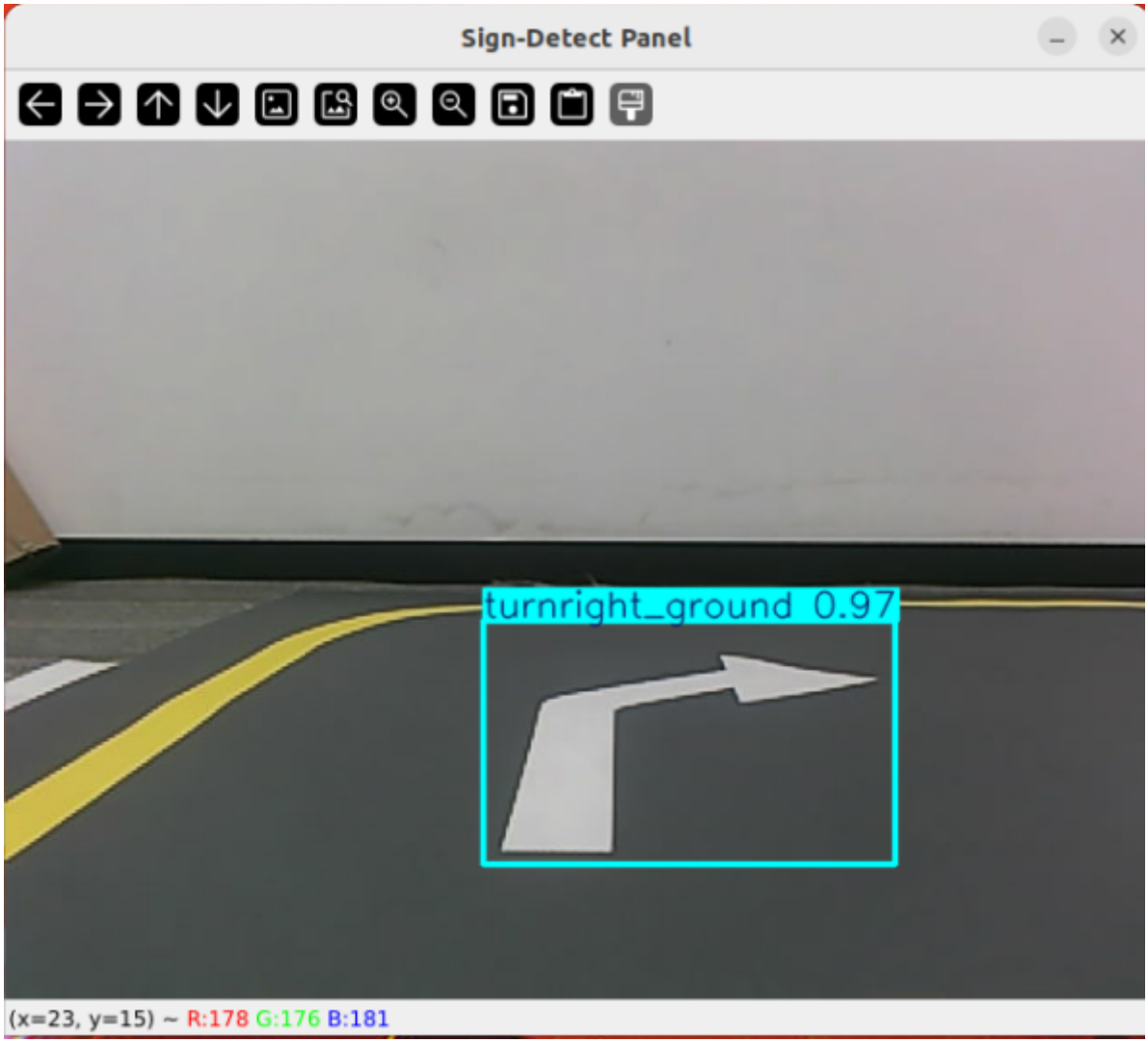
3.1 Startup Command

- Start the camera + chassis + road signs and lane markings

```
ros2 launch auto_drive auto_drive_bringup.launch.py
```

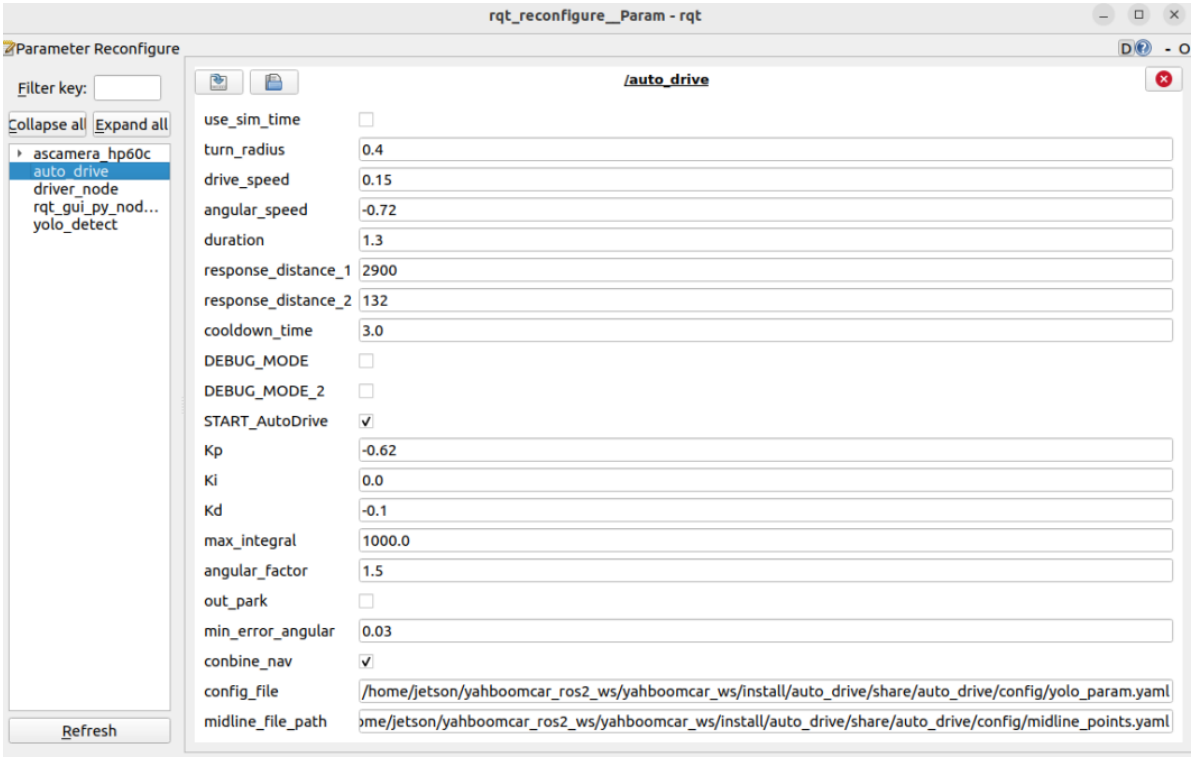


After the program runs, a YOLO recognition screen will be displayed. The car will move centered within the lane and perform corresponding actions based on the road signs it detects during its journey. (If the terminal does not report any errors after starting the program, but the car does not move, you can run the following program to check the parameters.)



- Start the dynamic parameter configuration node

```
ros2 run rqt_reconfigure rqt_reconfigure
```

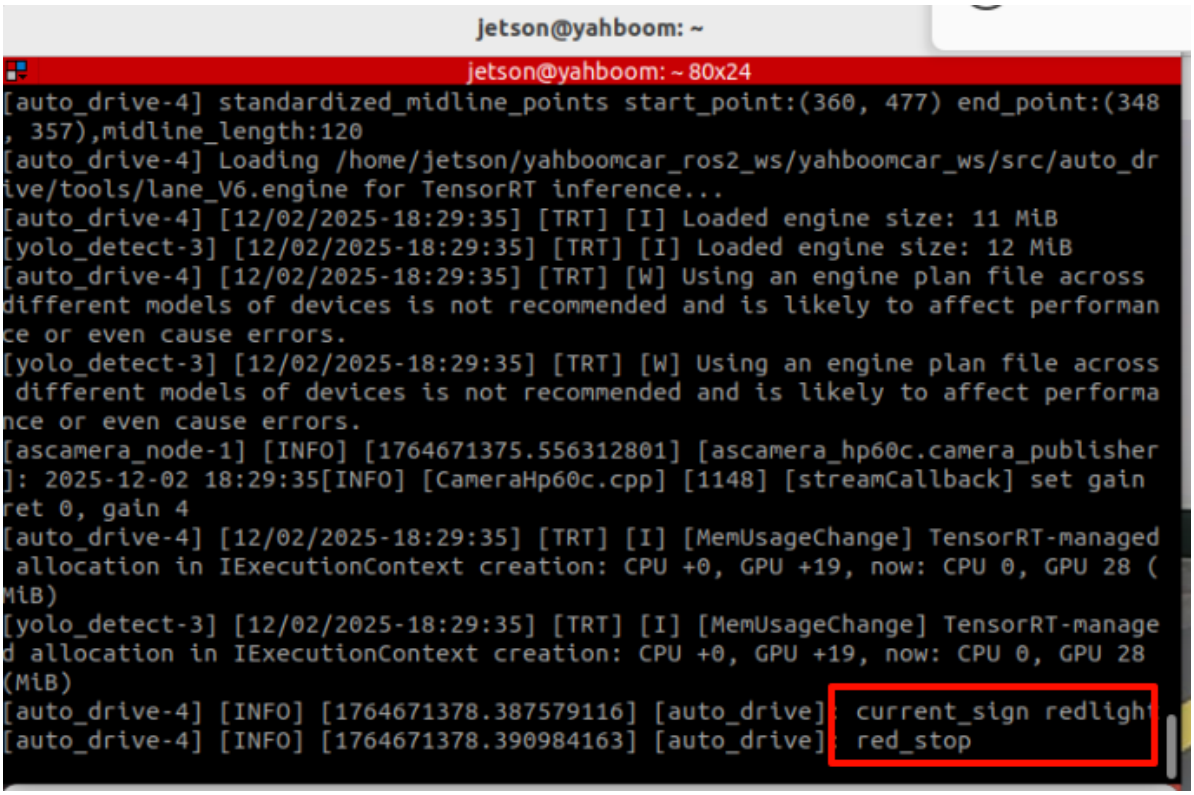


If the car doesn't move, check two parameters: whether START_AutoDrive is checked and whether conbine_nav is false.

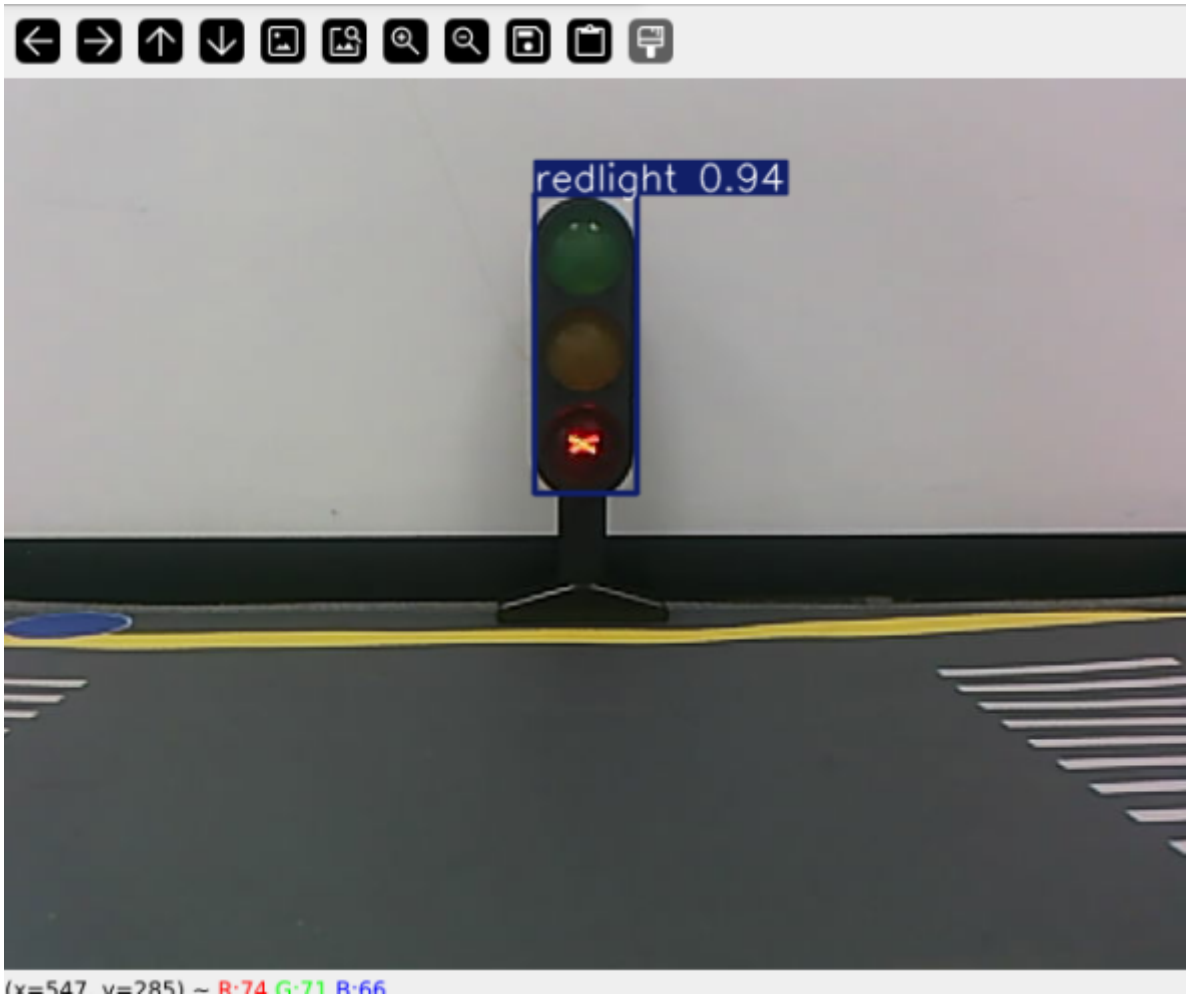
☑ After modifying the parameters, click on the blank space in the GUI to enter the parameter values. Note that this only applies to the current startup; for permanent effects, the parameters need to be modified in the source code.

3.2 Car Movement After Recognizing Traffic Lights

After the program recognizes a red light, it will print:



The car will remain stopped at the red light, waiting for it to turn green.

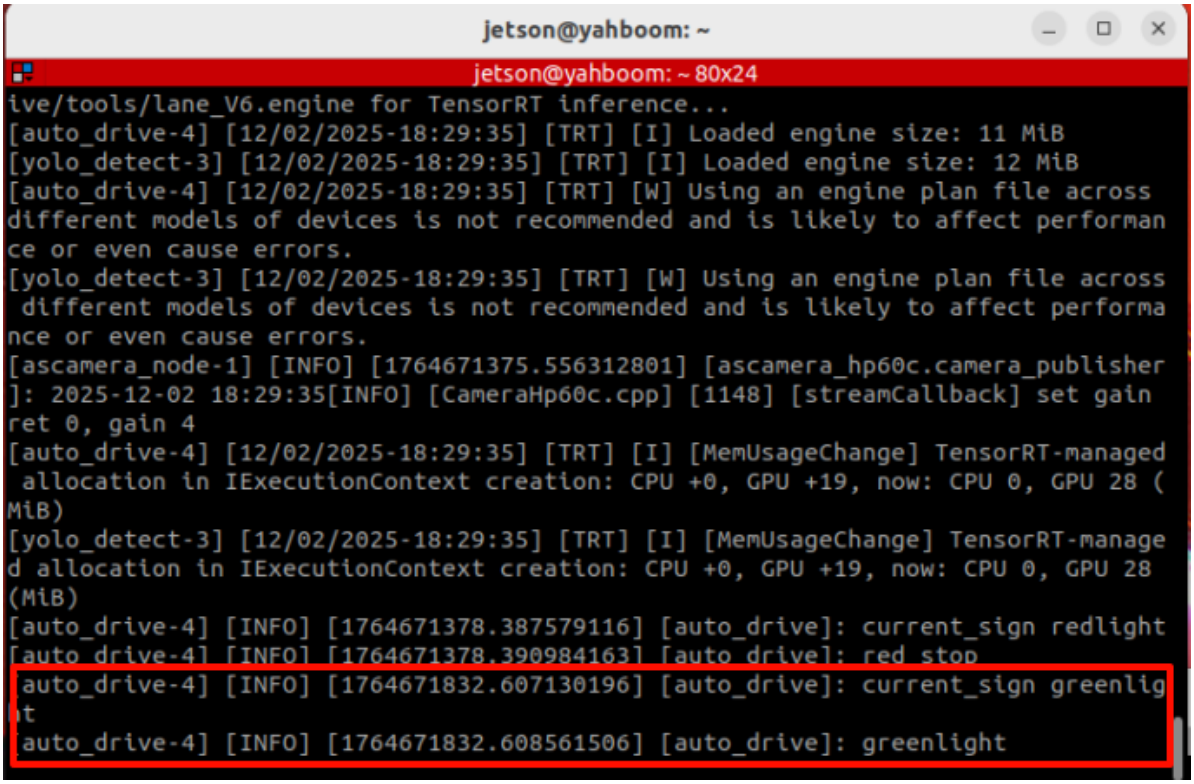


The red light handling function is located in the auto_drive.py file. Below is the function entered after recognizing a red light; users can modify the recognized actions themselves.

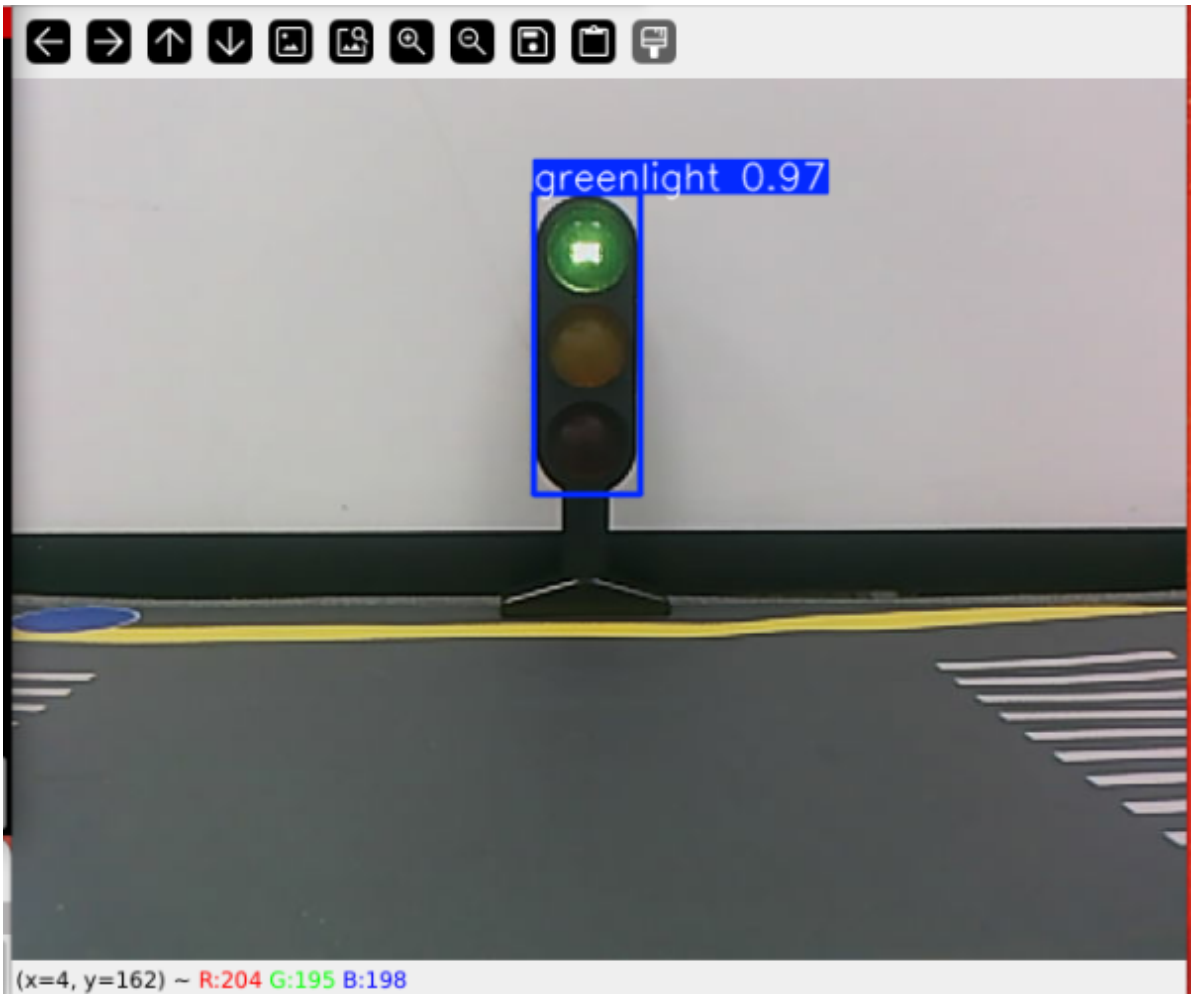
```
def red_stop(self):
        '''停止 stop'''
        if self.enable_route_nav:
            self.cancel_nav_pub.publish(Bool(data=True))
        else:
            self.get_logger().info('red_stop')
            self.turn_action_active = True  # Set an action sign and pause lane
control.
            self.__set_current_speed(linear_x=0.0, angular_z=0.0)
            time.sleep(3)
```

This section determines whether the car is using a road network for navigation. If it isn't, the car will pause lane control and stop to wait for another sign. If it is using a road network for navigation, it will pause the current navigation.

After recognizing a green light, the program will print:



The car will then release from the parking state and move forward at its initial speed.



The green light handling function is located in the `auto_drive.py` file. Below is the function executed after recognizing a green light. Users can modify the function to see how the car moves after recognizing a road sign.

```
def greenlight(self):
        '''绿灯greenlight'''
        self.get_logger().info('greenlight')
       if self.enable_route_nav and self.recentest_pose is not None:
            self.road_net_nav_pub.publish(self.recentest_pose)

        else:
            self.turn_action_active = False
            self.__set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
```

If the car is currently using a road network for navigation, it will republish the navigation status it was in before stopping to allow it to continue navigation. If it's not using a road network, it will restart lane-based navigation.

4. Source Code Analysis

4.1 auto_drive.py

YOLO road sign message reception:

```
def sign_callback(self, msg:DetectionObject):
    '''Subscribe to YOLO recognition results'''
    class_name = msg.class_name
    object_distance = self._get_distance(msg.ymin)  # Calculate distance
    object = Object(msg.class_name, msg.confidence, object_distance)
```

Multiple filtering mechanisms: mainly to prevent continuously recognizing road signs and triggering actions; the action will only be unlocked when different road signs are recognized.

```
# 1. Check if other actions are being performed.
if self.action_runing:
    return

# 2. Check cooldown time (to prevent repeated triggering)
current_time = time.time()
if class_name in self.last_processed_time:
    time_diff = current_time - self.last_processed_time[class_name]
    if time_diff < self.cooldown_time:  # Default cooldown: 3 seconds
        return

# 3. Check if it is a duplicate road sign.
if class_name == self.last_sign:
    return

# 4. Check if the road sign is enabled.
if not self.sign_enable_list[对应的索引]:
    return
```

Event queue processing

```
def handle_events(self):
    '''Handling target recognition events'''
    while True:
        if not self.event_queue.empty():
```

```
            object = self.event_queue.get()
            self.action_runing = True
            current_sign = object.class_name

            try:
                # Perform the appropriate action based on the type of road sign.
                if object.class_name == "straight" and
self.sign_enable_list[10]:
                    self.go_straight()
                elif object.class_name == "turnright" and
self.sign_enable_list[11]:
                    self.turn('right')
                # ... Other road sign processing

            finally:
                # Update status record
                self.last_processed_time[current_sign] = time.time()
                self.last_sign = current_sign
                self.action_runing = False
```

Specific action implementation

```
def greenlight(self):
        '''绿灯greenlight'''
        self.get_logger().info('greenlight')
        if self.conbine_nav and self.recentest_pose is not None:
            self.road_net_nav_pub.publish(self.recentest_pose)

        else:
            self.turn_action_active = False

def red_stop(self):
        '''停止 stop'''
        if self.conbine_nav:
            self.cancel_nav_pub.publish(Bool(data=True))
        else:
            self.get_logger().info('red_stop')
            self.turn_action_active = True  # Set an action sign and pause lane
control.
            self.__set_current_speed(linear_x=0.0, angular_z=0.0)
            time.sleep(3)
```